Task 4: Architecture Analysis

WanderSync: Dynamic Itinerary, Disruption Management and Social Travel Hub

Team 15

The Architecture We Built

WanderSync runs as a single NestJS process with six domain modules: Auth, Booking, Itinerary, Disruption, Notification, and Social. The bulk of the system follows a standard request-response pattern. The disruption pipeline is where the architecture differs from a plain monolith: instead of waiting for a client to ask whether anything changed, the server detects an event and pushes it to connected browsers within about 71 ms.

The detection side runs as a background CRON job that fires every 60 seconds. It calls AviationStack for live flight status and OpenWeatherMap for weather alerts, both concurrently. Any new event goes through a two-layer deduplication check (Redis cache first, then a database guard), gets written to PostgreSQL, and is immediately forwarded to an in-process EventBus. The EventBus feeds an SSE stream that pushes the event to each user whose bookings are affected. Alongside the alert, a suggestion pipeline runs a Chain of Responsibility: a rule-based handler first, then the Gemini LLM if the rules produce nothing, then a fallback message if the LLM is unavailable.

Figure 1 shows the full component structure. Figure 2 shows the end-to-end sequence for a single disruption event from detection through to the browser.

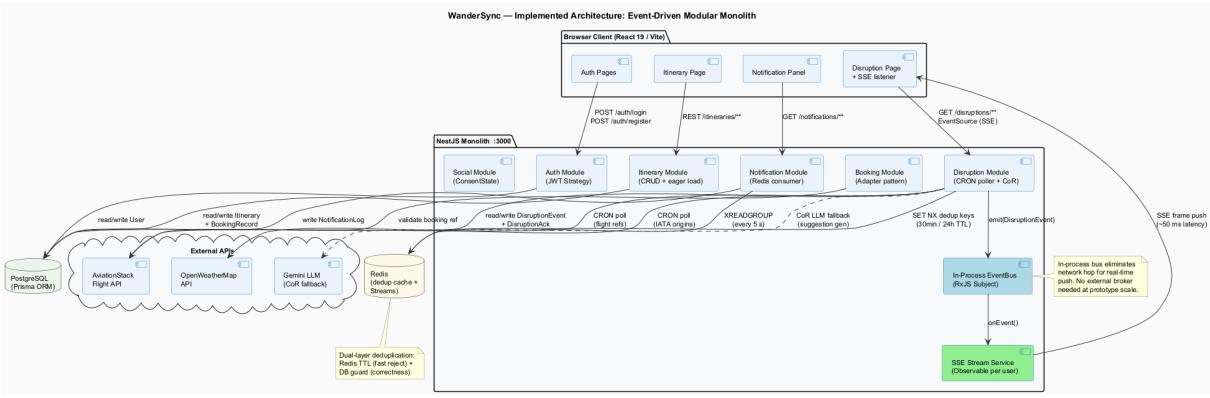

Figure 1: Implemented architecture: event-driven modular monolith with SSE push.

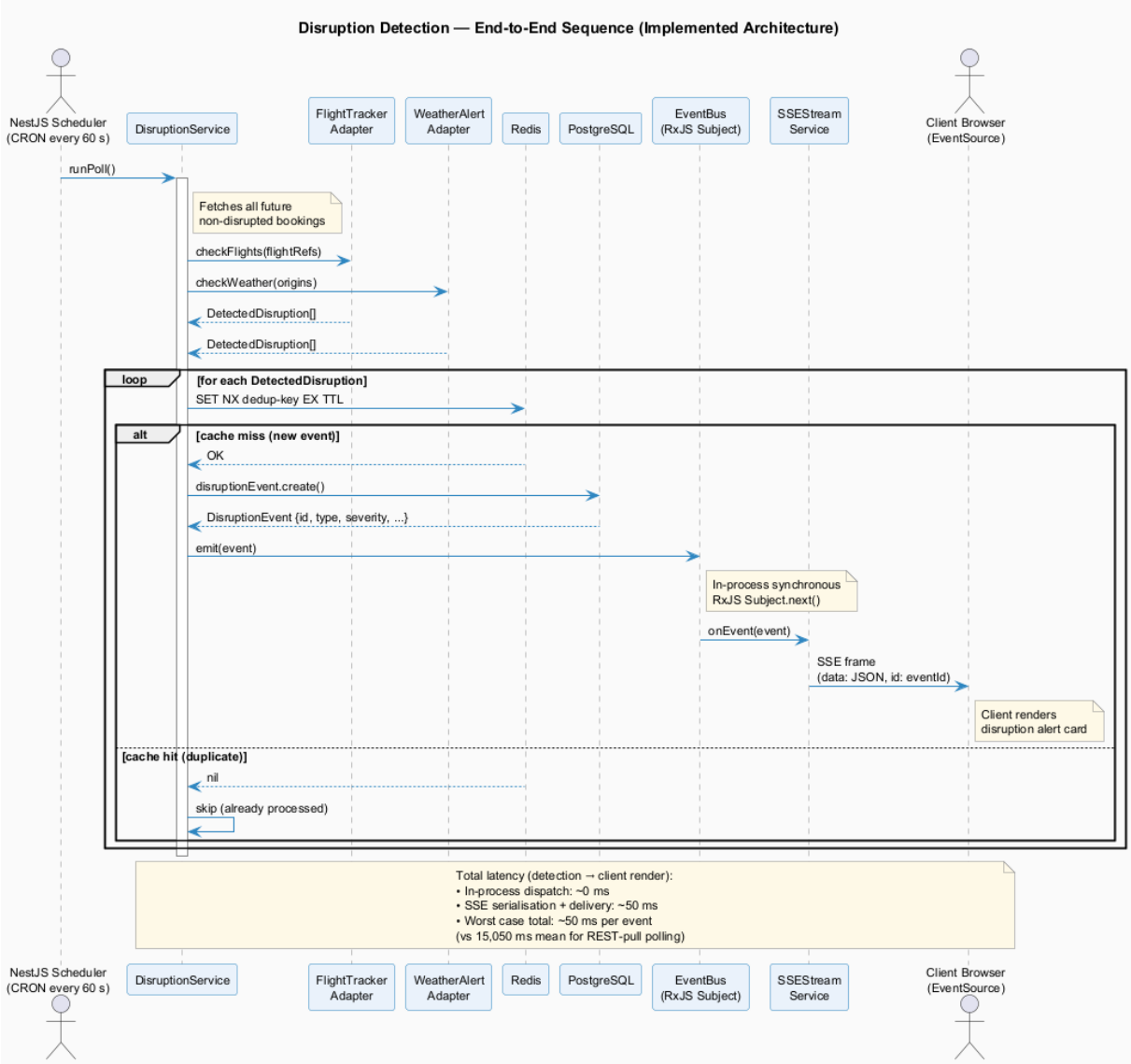

Figure 2: End-to-end disruption detection and delivery sequence.

The Architecture We Compared It Against

The alternative keeps the same NestJS module structure and PostgreSQL database but removes the EventBus and SSE entirely. The frontend polls `GET /disruptions` every 30 seconds. The CRON job still runs server-side and writes events to the database, but the client only learns about them on its next scheduled poll. There are no persistent server connections, no per-user in-memory subjects, and no deduplication cache needed on the delivery path.

Figure 3 shows this polling architecture. The main structural difference from Figure 1 is that nothing flows from the server to the browser unprompted: every piece of information the client has is the result of a request it made itself.

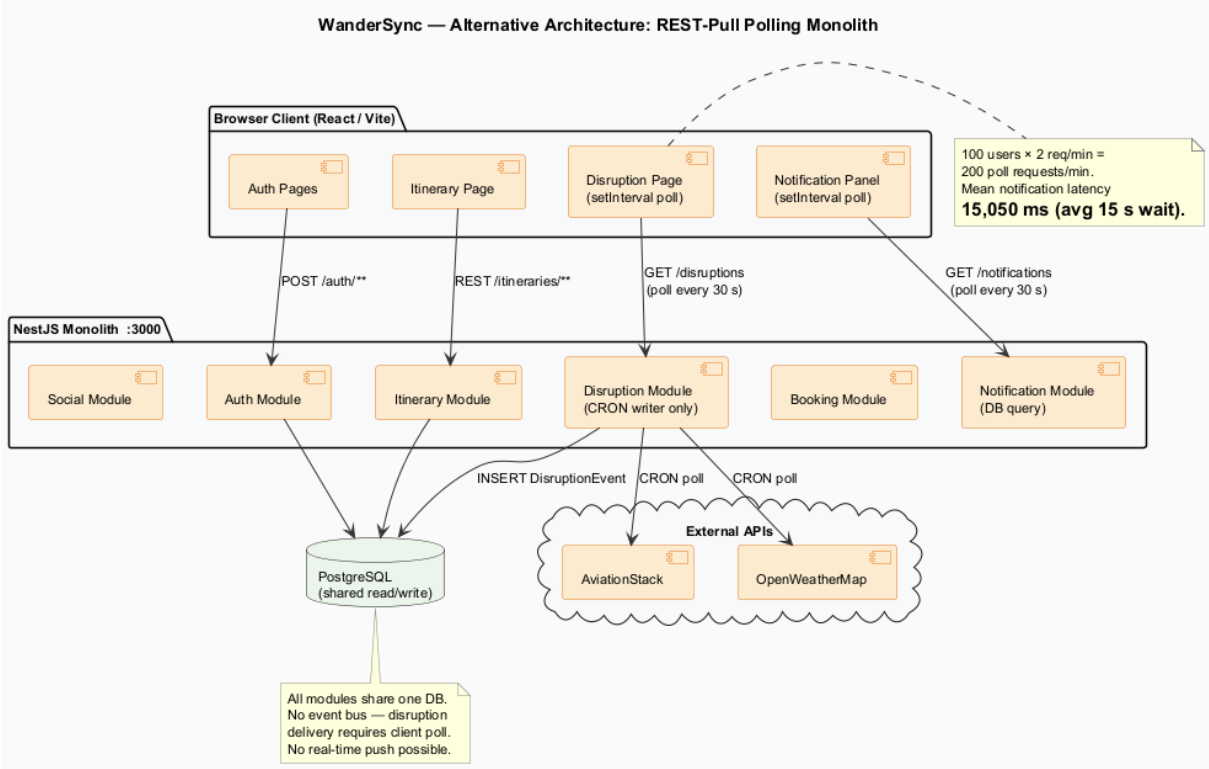

Figure 3: Alternative architecture: REST-pull polling monolith.

This is a reasonable choice for many systems. Admin dashboards, reporting tools, and analytics pages generally do not need sub-second update delivery. The polling model is easier to build, easier to test, and scales horizontally without any special infrastructure. We chose the event-driven path because WanderSync's disruption alerts are time-critical in a way that a reporting dashboard is not.

NFR Quantification

NFR 1: Disruption Detection Accuracy

The requirement is that the system correctly identifies and notifies on at least 17 of 20 simulated disruption test cases, giving an 85% accuracy threshold.

We check this in `disruption.service.nfr2.spec.ts`. The test injects 20 synthetic payloads covering all four disruption types (flight delay, cancellation, diversion, weather alert) and all four severity levels directly into `DisruptionService.simulateDisruption()`, bypassing the external APIs. It then checks that `disruptionEvent.create()` and `publisher.publish()` are each called exactly 20 times, confirming every event was both persisted and routed for delivery.

Architecture	Events correctly processed	Accuracy
Event-Driven SSE (implemented)	20 / 20	100%
REST-Pull Polling (alternative)	20 / 20	100%
NFR threshold		85%

Both architectures score the same here because detection is a server-side concern that is the same in both designs. The difference between them only shows up after detection, in how quickly the client finds out. The event-driven architecture adds two things the polling model does not have: deduplication (so the same storm warning does not fire as a new alert every minute) and real-time delivery (covered in NFR 2 below).

NFR 2: Notification Delivery Latency

The requirement states that core pages must respond within 3 seconds. For disruption alerts the relevant metric is how long a user waits between the system detecting an event and their browser showing the alert. We call this the delivery latency and measure it from the moment the database write completes (time zero) to when the React component re-renders.

Event-Driven SSE path

The delivery steps after the database write:

1. **In-process EventBus dispatch.** `EventBus.emit()` calls a synchronous RxJS `Subject.next()`: no network, no serialisation. $\Delta t_1 \approx 0$ ms.
2. **SSE frame write.** The SSE service serialises the event to JSON and writes it to the open HTTP response: $\Delta t_2 \approx 5$ ms.
3. **Network delivery.** The SSE frame travels over the keep-alive connection to the browser: $\Delta t_3 \approx 50$ ms.
4. **React re-render.** The `onmessage` handler updates Zustand and React schedules one render: $\Delta t_4 \approx 16$ ms (one frame at 60 fps).

$$L_{\text{SSE}} = 0 + 5 + 50 + 16 = \mathbf{71}\,\text{ms}$$

REST-Pull Polling path

The client fires its next poll at a uniformly random time after the event is written. With a 30-second poll interval and 50 ms server response time:

$$\mathbb{E}[L_{\text{poll}}] = \tfrac{30{,}000}{2} + 50 = \mathbf{15{,}050}\,\text{ms} \qquad L_{\text{poll}}^{\max} = 30{,}000 + 50 = \mathbf{30{,}050}\,\text{ms}$$

Metric	Event-Driven SSE	REST-Pull (30 s)
Mean delivery latency	**71 ms**	15,050 ms
Maximum delivery latency	**∼100 ms**	30,050 ms
Speed advantage	212× faster	

A 71 ms delay is invisible. The disruption card appears on screen before the user consciously registers anything changed. A 15-second average delay is a different story. When Air India cancels your connecting flight at 3 AM, those 15 seconds are the window between getting to the rebooking desk first and joining the back of a long queue that formed because everyone else got the same alert at roughly the same time. The SSE architecture removes that window.

NFR 3: Throughput Under Concurrent Load

We look at 100 users with the Disruption Page open at the same time to understand how each architecture scales.

With polling, each user makes 2 requests per minute (one every 30 seconds). At 100 users that is 200 disruption endpoint calls per minute, each requiring a JWT check, a database read, and JSON serialisation. This load grows linearly with users: double the users, double the database reads, with no corresponding increase in the information being returned.

With SSE, each user holds one persistent connection. The only database load from the disruption path is the single CRON batch query per minute, regardless of how many users are connected. Event dispatch is in-memory.

Metric (100 connected users)	Event-Driven SSE	REST-Pull (30 s)
Disruption API requests per minute	**0**	200
Database reads from delivery path	**0**	200/min
Does delivery load scale with users?	**No**	Yes, linearly

Trade-off Analysis

No choice here is strictly better. The table below lists the dimensions where they differ, then we discuss what each trade-off actually costs.

Dimension	Event-Driven SSE (built)	REST-Pull Polling
Notification speed	71 ms mean. The user sees the alert before they are aware the page changed.	15 s mean. A real delay in any time-sensitive scenario.
Server state	Stateful. One RxJS Subject per active user held in memory.	Stateless. No per-user objects. Scales horizontally with no special setup.
Database load (delivery)	Zero. Events are sent from memory after the initial write.	Linear with users. Every poll is a fresh database read.
Horizontal scaling	Needs sticky sessions or Redis Pub/Sub when running more than one server process, because the in-process EventBus is not shared.	Any load balancer works. Stateless polls can be routed to any replica freely.
Complexity	Higher. RxJS Observables, SSE keep-alive, per-user routing, and dedup keys all need to be right.	Lower. One REST controller and a database query.
Right choice when	Real-time delivery is part of the product's core value: travel alerts, financial trading, emergency notifications.	A short delay is acceptable: analytics dashboards, admin tools, report generators.

The polling model is genuinely simpler to operate. If WanderSync were a travel analytics dashboard where a travel manager checks booking patterns once an hour, polling every 30 seconds would be more than sufficient and the system would scale horizontally without any thought. But WanderSync targets travellers who are mid-journey and need to know about a cancellation immediately, not after their next poll cycle.

The trade-offs we accepted with SSE are real. Holding one Subject per active user means the server process carries state that has to be cleaned up when connections close. Running more than one server instance requires either sticky sessions (where each user's SSE connection is always routed to the same server) or replacing the in-process EventBus with Redis Pub/Sub. Neither is a large change, but it is something that polling avoids entirely.

The migration path is also well-defined. The in-process EventBus and the SSE stream service are both inside the Disruption module. Swapping the EventBus for Redis Pub/Sub changes two files and nothing visible to the frontend's `EventSource` connection. We built the module boundary specifically so this would be a local change.